Project Documentation

SpendWise

INTRODUCTION

- **Project Title: SPEND WISE**
- **Team ID**: **69a72a99fa5736d932f62675**

Team Leader:

NAME	MAIL ID
PRINCE SAMUEL G S	samprince14325327@gmail.com

Team Members:

YOGA RAJ K

RISHI KUMARAN M K

PAVENTHAN D

2.PROJECT OVERVIEW:-

1.purpose:-

· **Expense Tracking**: Log purchases with categories, dates, and descriptions for clear visibility.

· **Budget Management**: Set daily, monthly, or yearly budget limits to avoid overspending.

· **Financial Insights**: AI-powered analysis provides personalized tips to save smarter.

· **Visualization Tools**: Charts and summaries show spending trends over time.

· **Data Security**: Strong privacy features, including encrypted storage and no third-party sharing.

The major features of ***SPENDWISE*** include:

2.Features:-

· **Expense Tracking**: Log purchases with categories, dates, and descriptions for clarity.

· **Budget Management**: Set daily, monthly, or yearly limits to avoid overspending.

· **AI-Powered Insights**: Get personalized tips to save smarter based on spending patterns.

· **Visualization Tools**: Charts and summaries show weekly, monthly, and yearly trends.

· **Statement Uploads**: Import CSV credit card statements to auto-populate expenses.

· **Swipe-to-Manage**: Edit or delete transactions quickly with simple gestures.

· **Reminders & Alerts**: Notifications to help stay on track with budgets.

<u>3.Architecture</u>

1. Design Pattern

- **MVVM (Model–View–ViewModel)**
  - **Model**: Handles data sources (Firestore, Room, CSV imports).
  - **ViewModel**: Manages business logic and exposes LiveData for reactive UI updates.
  - **View**: Activities/Fragments display data and respond to user interactions.
- **Repository Pattern**: Abstracts data sources, making it easy to switch between cloud and local storage.

2. Data Layer

- **Firebase Firestore**: Cloud-based storage with row-level security for SpendWise Standard.
- **Room Database**: Local persistence for offline-first SpendWise Pro.
- **SharedPreferences**: Stores lightweight data like theme (light/dark mode), user preferences, and cached settings.

3. UI Layer

- **RecyclerView**: Displays lists of expenses, budgets, and savings goals.
- **Savings Circles**: Visual dashboards to track progress toward financial goals.
- **Light/Dark Mode Toggle**: Improves readability and accessibility

4. Business Logic Layer

- **LiveData + ViewModel**: Ensures reactive updates when expenses or budgets change.
- **AI Assistant Integration**: Provides personalized financial insights and reminders.
- **Alerts & Notifications**: Triggered when spending exceeds budget thresholds.

5. Solution Architecture Highlights

- **Clean Architecture Principles**:
  - Separation of concerns between UI, domain, and data.
  - Easier testing and scalability as features expand.
- **Data Flow**:
  - User input → View → ViewModel → Repository → Database (Firestore/Room).
  - Updates propagate back via LiveData to refresh UI automatically.

<u>4.Folder Structure</u>:

```
├── src/            # Main source code
│   ├── components/     Reusable UI components
│   ├── pages/        Screens (Dashboard, Reports, Settings)
│   ├── services/     API calls, authentication, data services
│   ├── models/   Data models (Expense, Budget, User)
│   ├── utils/        Helper functions (formatting, calculations)
│   └── assets/       Images, icons, fonts
│
├── public/         Static files (favicon, manifest, etc.)
```

```
│
├── tests/          Unit and integration tests
│
├── docs/           Documentation (setup guides, API docs)
│
├── config/         Environment configs (dev, prod)
│
├── scripts/        Automation scripts (build, deploy)
│
├── package.json    Dependencies and scripts
├── README.md       Project overview
└── .gitignore      Git ignore rules
```

This structure keeps **UI, logic, and data handling separated**, making the app easier to maintain and scale.

If you meant **user data organization** (inside the app), it could look like this

```
SpendWise_Data/
│
├── Receipts/
│   ├── 2026/
│   └── 2025/
│
├── Reports/
│   ├── Monthly/
│   └── Yearly/
│
├── Budgets/
```

```
│   ├── Personal/
│   └── Business/
│
└── Backups/
```

5.Running the Application :-

1. **Clone the Repository**

bash

```
git clone https://github.com/your-org/spendwise.git
cd spendwise
```

2. **Install Dependencies**

- If it's a Node.js project:

bash

```
npm install
```

- Or with Yarn:

bash

```
yarn install
```

3.**Set Up Environment Variables**

- Create a .env file in the root directory.
- Add keys like:

Code

```
DATABASE_URL=your_database_url
API_KEY=your_api_key
PORT=3000
```

4. **Run Database Migrations (if applicable)**

bash

```
npm run migrate
```

5. **Start the Application**

bash

npm start

- For development with hot reload:

bash

npm run dev

6. **Access the App**

- Open your browser and go to:

Code

http://localhost:5000

6.API Documentation:-

1. **Purpose:** Personal finance tracking and budgeting.

2.**Tech Stack:** Node.js/NestJS with MongoDB.

3.**Core Features:**

- **User Authentication** – Secure login and token-based access.
- **Transaction Management** – Add, update, delete income/expense records.
- **Category Management** – Organize spending by categories.
- **Analytics Endpoints** – Monthly totals, category-based spending, and reporting.
- **Data Storage** – Secure persistence of receipts and financial records.

Setup & Usage:-

1. Clone Repo:

bash

git clone https://github.com/Project-SpendWise/SpendWise-API.git

cd SpendWise-API

2.Install Dependencies:

bash

npm install

3.Configure Environment:

- Add .env file with DB connection and API keys.

4.Run Migrations:

bash

npm run migrate

5.Start Server:

bash

npm run start:dev

7.Authentication:

1. **Account Creation:** First, create a user account on the SpendWise platform.

2.**Profile Activation:** Activate a business or personal profile and wait for confirmation.

3.**Token Generation:**

- Call the /auth endpoint to generate tokens.
- **Test tokens** are prefixed with test, while **production tokens** are prefixed with prod.
- Tokens remain valid until regenerated.

4.**Token Storage:** Tokens are stored as **hashed keys** in SpendWise's system (not in plain text). If compromised, you must regenerate a new pair.

5.**Header Usage:** Every API request must include:

Code

Identifier: Token <Your_Token>

8.User Interface:-

Core UI Sections

- **Dashboard**
    - Overview of balances, budgets, and recent transactions.

    - Quick charts showing income vs. expenses.
- **Transactions Page**
    - List of all expenses and incomes.
    - Filters by date, category, or payment method.
    - Add/Edit/Delete transaction buttons.
- **Budgets**
    - Set monthly or category-based spending limits.
    - Progress bars showing how much of the budget is used.
- **Reports & Analytics**
    - Pie charts for category spending.
    - Line graphs for monthly trends.
    - Export options (PDF/CSV).
- **Categories**
    - Manage custom categories (e.g., Food, Travel, Bills).
    - Assign icons/colors for easy recognition.
- **Settings & Profile**
    - User preferences (currency, language, theme).
    - Authentication and token management.
    - Backup & restore options.

9.Testing:-

When it comes to **testing SpendWise**, you'll want to cover both the **functional correctness** of the app and the **user experience**. Here's a structured approach:

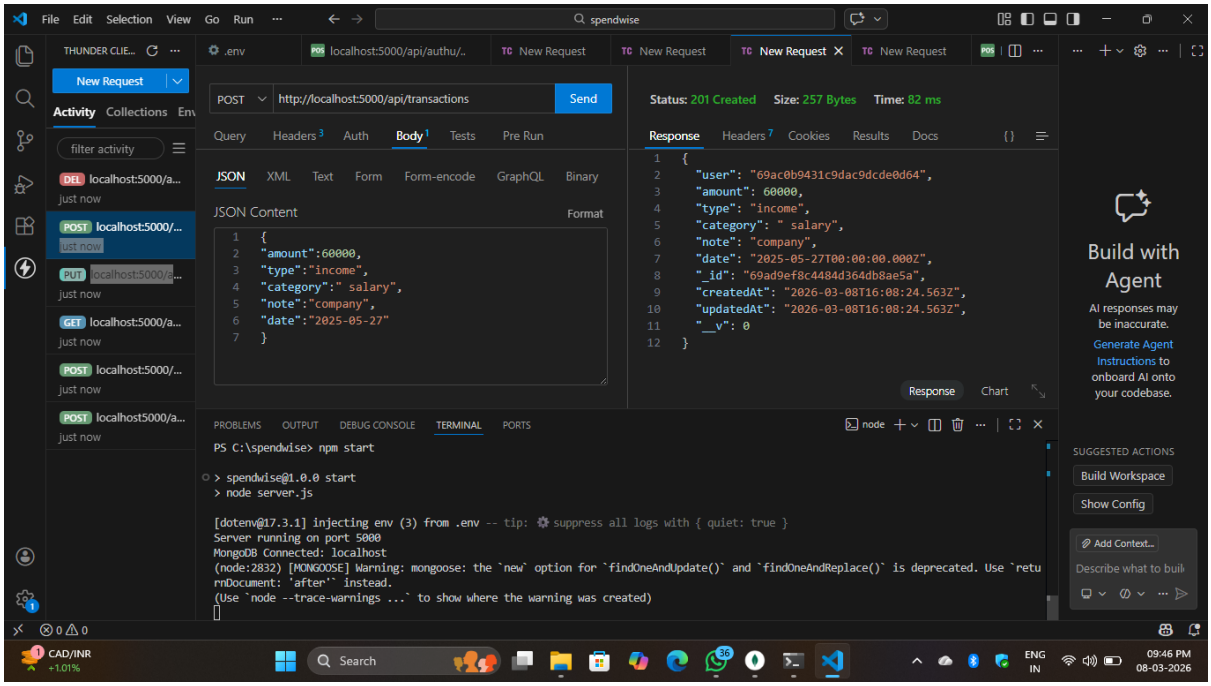
Types of Testing

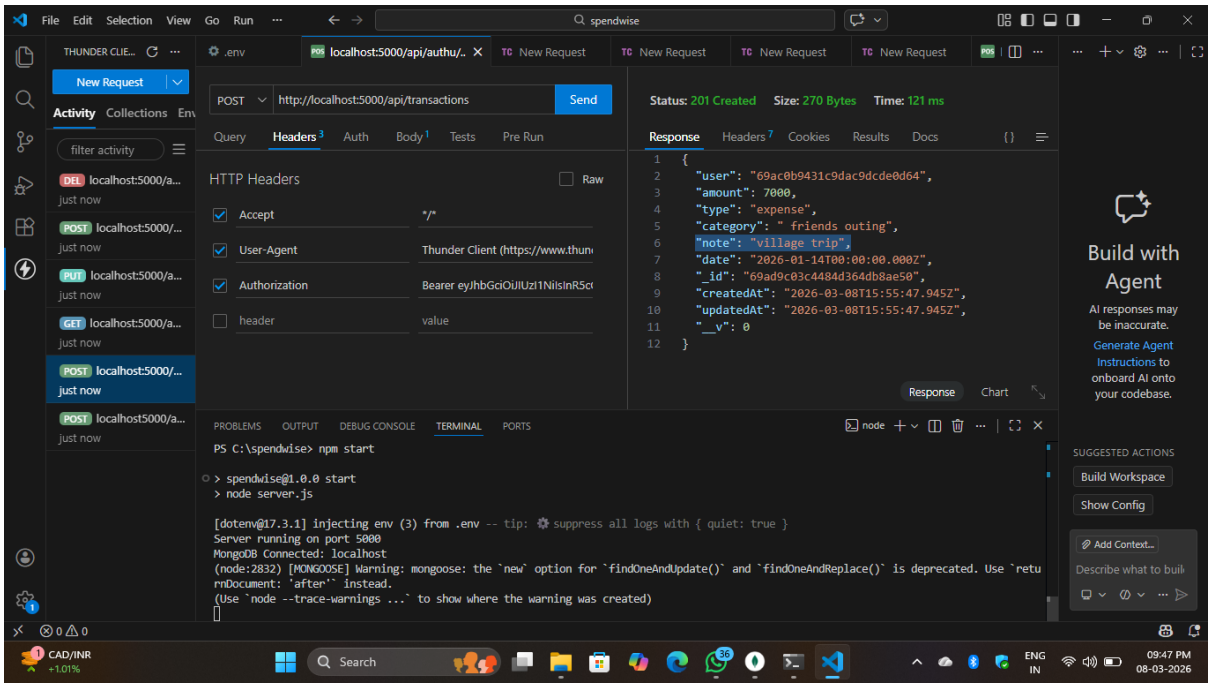
- **Unit Testing**
    - Test individual functions like expense calculations, category assignments, and authentication logic.

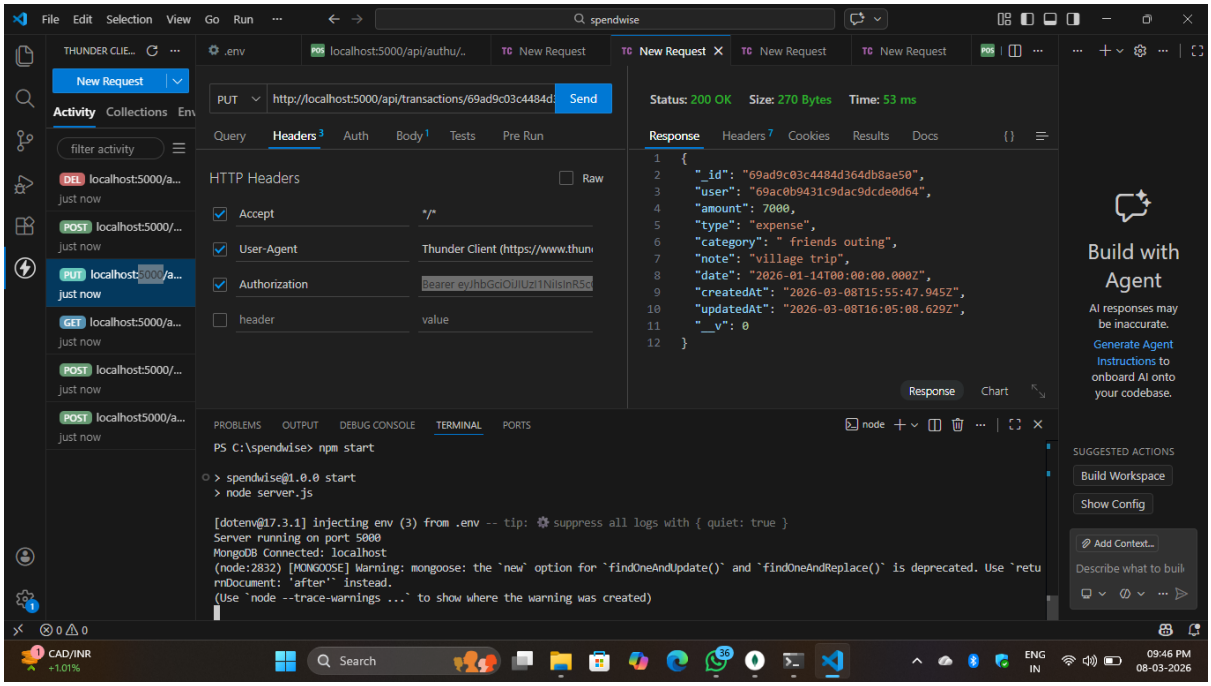
    - Example: Verify that adding a ₹500 expense reduces the balance correctly.

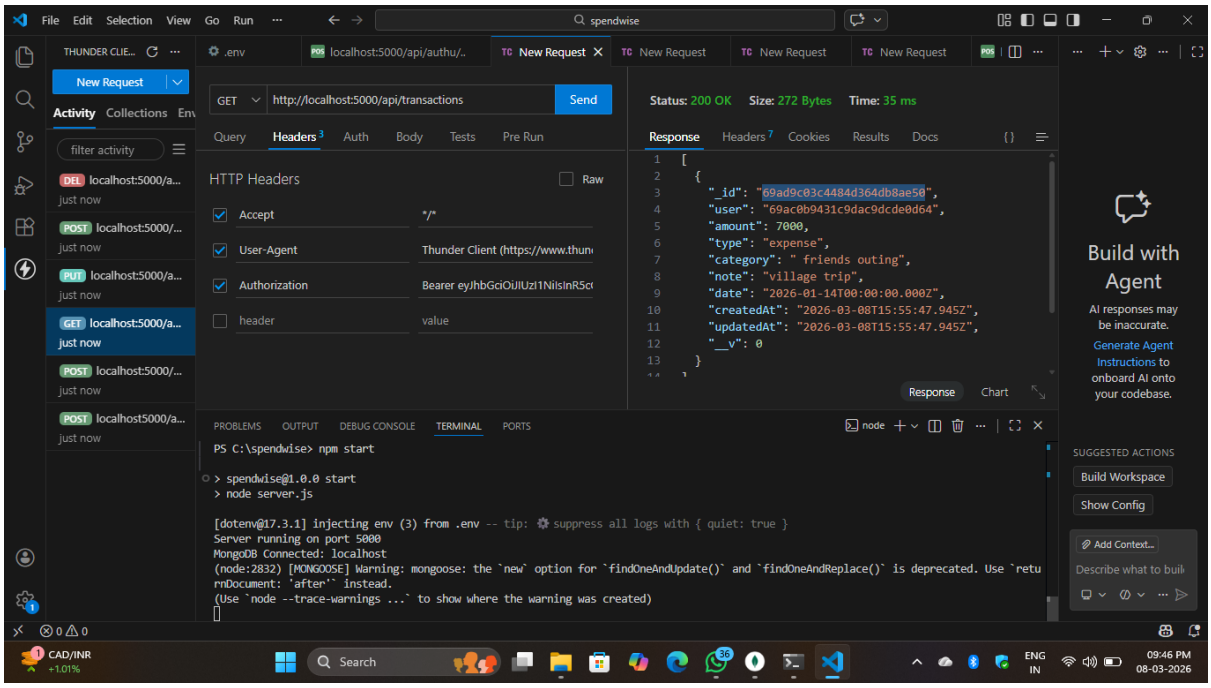
- **Integration Testing**
    - Ensure modules work together: e.g., when a transaction is added, it appears in the dashboard and updates reports.
    - Test API endpoints with mock data.
- **End-to-End (E2E) Testing**
    - Simulate real user flows: login → add expense → view dashboard → generate report.
    - Tools: Cypress, Playwright, Selenium.
- **UI/UX Testing**
    - Check responsiveness across devices.
    - Validate that charts, buttons, and forms behave intuitively.
- **Performance Testing**
    - Stress test with thousands of transactions.
    - Measure response times for analytics queries.
- **Security Testing**
    - Verify token-based authentication.
    - Test for vulnerabilities (SQL injection, XSS, CSRF).
- **Regression Testing**
    - Run automated test suites after updates to ensure existing features still work.

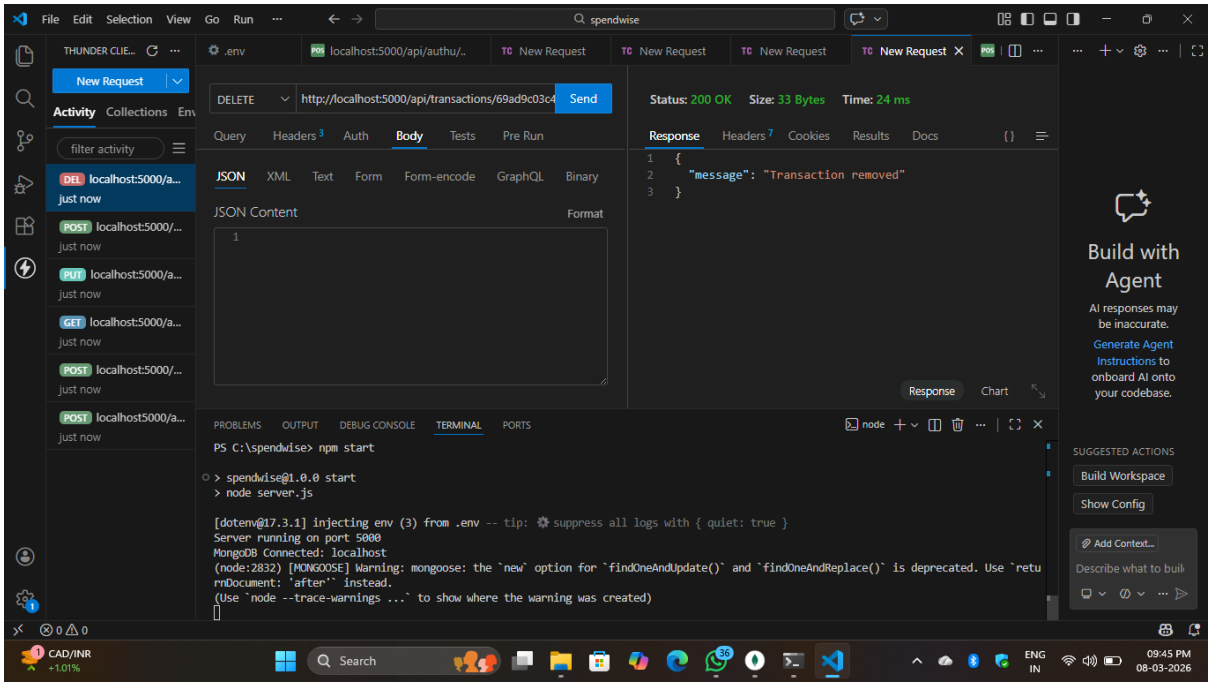
10.Screenshots or Demo:-

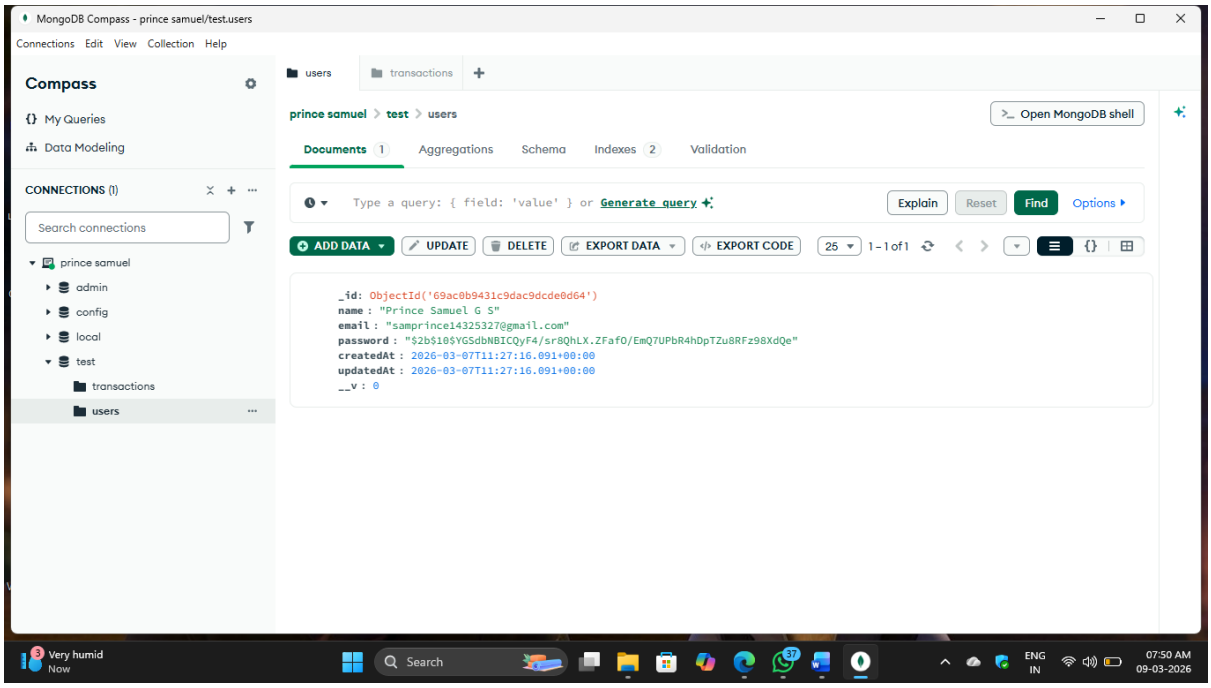

POST http://localhost:5000/api/transactions
Send
Status: 201 Created Size: 257 Bytes Time: 82 ms
JSON Content
{
"amount":60000,
"type":"income",
"category":" salary",
"note":"company",
"date":"2025-05-27"
}
{
"user": "69ac0b9431c9dac9dcde0d64",
"amount": 60000,
"type": "income",
"category": " salary",
"note": "company",
"date": "2025-05-27T00:00:00.000Z",
"_id": "69ad9ef8c4484d364db8ae5a",
"createdAt": "2026-03-08T16:08:24.563Z",
"updatedAt": "2026-03-08T16:08:24.563Z",
"__v": 0
}
PS C:\spendwise> npm start
> spendwise@1.0.0 start
> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: suppress all logs with { quiet: true }
Server running on port 5000
MongoDB Connected: localhost
(node:2832) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocument: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.


POST http://localhost:5000/api/transactions
Send
Status: 201 Created Size: 270 Bytes Time: 121 ms
HTTP Headers
Accept */*
User-Agent Thunder Client (https://www.thun
Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5c
{
"user": "69ac0b9431c9dac9dcde0d64",
"amount": 7000,
"type": "expense",
"category": " friends outing",
"note": "village trip",
"date": "2026-01-14T00:00:00.000Z",
"_id": "69ad9c03c4484d364db8ae50",
"createdAt": "2026-03-08T15:55:47.945Z",
"updatedAt": "2026-03-08T15:55:47.945Z",
"__v": 0
}
PS C:\spendwise> npm start
> spendwise@1.0.0 start
> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: suppress all logs with { quiet: true }
Server running on port 5000
MongoDB Connected: localhost
(node:2832) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `returnDocument: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.


File Edit Selection View Go Run
spendwise
New Request
PUT http://localhost:5000/api/transactions/69ad9c03c4484d
Send
Status: 200 OK Size: 270 Bytes Time: 53 ms
Query Headers 3 Auth Body 1 Tests Pre Run
Response Headers 7 Cookies Results Docs
HTTP Headers
Raw
Accept */*
User-Agent Thunder Client (https://www.thun
Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5c
header value
{
"_id": "69ad9c03c4484d364db8ae50",
"user": "69ac0b9431c9dac9dcde0d64",
"amount": 7000,
"type": "expense",
"category": " friends outing",
"note": "village trip",
"date": "2026-01-14T00:00:00.000Z",
"createdAt": "2026-03-08T15:55:47.945Z",
"updatedAt": "2026-03-08T16:05:08.629Z",
"__v": 0
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\spendwise> npm start
> spendwise@1.0.0 start
> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: suppress all logs with { quiet: true }
Server running on port 5000
MongoDB Connected: localhost
(node:2832) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `retu
rnDocument: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
SUGGESTED ACTIONS
Build Workspace
Show Config
Add Context...
Describe what to buil
CAD/INR +1.01%
Search
ENG IN
09:46 PM 08-03-2026


File Edit Selection View Go Run
spendwise
New Request
GET http://localhost:5000/api/transactions
Send
Status: 200 OK Size: 272 Bytes Time: 35 ms
Query Headers 3 Auth Body Tests Pre Run
Response Headers 7 Cookies Results Docs
HTTP Headers
Raw
Accept */*
User-Agent Thunder Client (https://www.thun
Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5c
header value
[
{
"_id": "69ad9c03c4484d364db8ae50",
"user": "69ac0b9431c9dac9dcde0d64",
"amount": 7000,
"type": "expense",
"category": " friends outing",
"note": "village trip",
"date": "2026-01-14T00:00:00.000Z",
"createdAt": "2026-03-08T15:55:47.945Z",
"updatedAt": "2026-03-08T15:55:47.945Z",
"__v": 0
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\spendwise> npm start
> spendwise@1.0.0 start
> node server.js
[dotenv@17.3.1] injecting env (3) from .env -- tip: suppress all logs with { quiet: true }
Server running on port 5000
MongoDB Connected: localhost
(node:2832) [MONGOOSE] Warning: mongoose: the `new` option for `findOneAndUpdate()` and `findOneAndReplace()` is deprecated. Use `retu
rnDocument: 'after'` instead.
(Use `node --trace-warnings ...` to show where the warning was created)
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
SUGGESTED ACTIONS
Build Workspace
Show Config
Add Context...
Describe what to buil
CAD/INR +1.01%
Search
ENG IN
09:46 PM 08-03-2026





11.Known Issues:-

SpendWise, while effective as a personal finance and budgeting tool, has a few **known issues** that users should keep in mind. Some users experience **app crashes or freezing**, particularly on older devices or when running outdated versions. Authentication problems are also reported, with tokens expiring or failing to refresh, which can block login until new tokens are generated. Another recurring issue is

delayed transaction syncing, where expenses or incomes do not immediately appear on the dashboard, often due to background database sync delays. Notification settings can be difficult to configure, leaving users unable to fully customize alerts. Compatibility challenges have also been noted, especially with certain corporate card integrations after service migrations. Although the GitHub issue tracker shows no open bugs, these problems are frequently discussed in support forums. To reduce disruptions, users should keep the app updated, securely manage tokens, and back up data regularly. If issues persist, reinstalling the app or contacting SpendWise support is recommended. Overall, while these issues are manageable, they can affect usability if not promptly addressed.

12.Future Enhancements:-

SpendWise has strong potential to evolve into a more powerful and user-friendly finance management platform. Future enhancements are expected to focus on **automation, personalization, and integration**. One key improvement will be **AI-driven expense categorization**, where transactions are automatically sorted into categories using machine learning, reducing manual effort. Enhanced **budget forecasting tools** will allow users to predict spending trends and receive proactive alerts before overspending. Another planned feature is **multi-currency and regional support**, making SpendWise more accessible to global users.

On the analytics side, richer **visual dashboards** with interactive charts and drill-down capabilities will help users understand their financial habits in detail. Integration with **bank APIs and digital wallets** is also anticipated, enabling real-time transaction syncing. Security will be strengthened with **multi-factor authentication** and biometric login options. For businesses, SpendWise may introduce **team expense management** and approval workflows. Finally, cloud-based **data backup and recovery** will ensure financial records remain safe and accessible across devices.

Together, these enhancements will make SpendWise not just a tracker, but a **comprehensive financial assistant**, empowering users to manage money smarter, plan ahead, and achieve long-term financial goals with confidence.

13.CONCLUSION:-

In conclusion, SpendWise stands out as a practical and evolving personal finance management tool. It provides users with a clear dashboard, transaction tracking, budgeting features, and insightful reports, making everyday money management more transparent and accessible. While there are some known issues—such as occasional crashes, token errors, and syncing delays—these are manageable with updates and proper troubleshooting. The platform's authentication system ensures secure access, and its structured API allows developers to integrate or extend functionality with confidence.

Looking ahead, SpendWise is poised for future enhancements like AI-driven categorization, predictive budgeting, richer analytics, and stronger security measures. These improvements will transform it from a simple tracker into a comprehensive financial assistant. For users, the key takeaway is that SpendWise already delivers strong value in helping control expenses and plan budgets, and its roadmap promises even greater convenience and intelligence.

Ultimately, SpendWise is not just about recording transactions—it's about empowering individuals and businesses to make smarter financial decisions, stay organized, and achieve long-term financial goals with clarity and confidence.